Learning Multimodal AI: The 20% That Matters

1 What "Multimodal" Means

A multimodal model handles more than one kind of data — text *plus* images, audio, or video — often in a single model. The most practical and common case today is the **Vision-Language Model (VLM)**: text + images. Over the past year multimodal moved from buzzword to baseline; models no longer stop at text but interpret images, audio, video, and even user interfaces, fusing perception with reasoning.

This document extends everything you learned about text models into other modalities. The good news: the core ideas (tokens, embeddings, retrieval, prompting) carry over almost unchanged.

2 The One Key Idea

Every modality gets converted into **tokens / embedding vectors in a shared space** that the model processes the same way. Text is already split into tokens (the Hugging Face document). An image is turned into a sequence of "visual tokens" and projected into the *same* embedding space as text, so the language model can read pixels and words in one stream. Once you internalize "images become tokens too," multimodal stops being mysterious.

3 How a VLM Works (Three Steps)

A vision-language model solves three sub-problems in sequence:

1. **Encode** — an image encoder (a Vision Transformer, ViT, often SigLIP) splits the image into patches and turns them into vectors.
2. **Align / connect** — a small "connector" (often an MLP projector) maps those image vectors into the language model's token-embedding space, so they look like tokens to the LLM.
3. **Generate** — the LLM receives the image tokens followed by your text prompt, and generates text as usual.

So a VLM is typically: **image encoder + connector + a normal LLM backbone**. Many open VLMs are built exactly this way (e.g. a SigLIP encoder bolted onto a Qwen language model).

4 CLIP and SigLIP (The Embedding Backbone)

CLIP trained an image encoder and a text encoder *together* on hundreds of millions of image–caption pairs using **contrastive learning**: matching image/text pairs are pushed close in a shared embedding space, mismatched ones pushed apart. The result is a space where a picture of a dog and the words "a dog" land near each other — enabling cross-modal search and zero-shot classification with no task-specific labels.

SigLIP is CLIP's successor and what most modern open VLMs use. CLIP's softmax loss needs huge batches to see enough negatives; SigLIP replaces it with a **sigmoid loss** that treats each image–text pair as an independent yes/no match, giving a lower memory footprint and strong results at smaller batch sizes.

Why you care: CLIP/SigLIP embeddings are how you put images and text in one vector space — the foundation of multimodal search and multimodal RAG below.

5 Using a Multimodal Model — Path A: Hosted API

The fastest route. Multimodal APIs accept images, usually as base64, alongside text. This reuses the API patterns you already know.

```python
import base64, requests

with open("chart.png", "rb") as f:
    img_b64 = base64.standard_b64encode(f.read()).decode()

# Shape of a multimodal message: a list mixing image and text parts
messages = [{
    "role": "user",
    "content": [
        {"type": "image", "source": {"type": "base64",
         "media_type": "image/png", "data": img_b64}},
        {"type": "text", "text": "What trend does this chart show?"},
    ],
}]
# POST to your provider's endpoint as in the API-calls document
```

Key point: the message content becomes a *list* of typed parts (image + text), not a plain string. That is the only real difference from a text-only call.

6 Using a Multimodal Model — Path B: Open VLM via Hugging Face

For self-hosting, privacy, or fine-tuning. The `AutoProcessor` is the multimodal version of the tokenizer: it preprocesses *both* the image and the text into model inputs.

```python
from transformers import AutoProcessor, AutoModelForVision2Seq
from PIL import Image
import torch

processor = AutoProcessor.from_pretrained("model-name")
model = AutoModelForVision2Seq.from_pretrained("model-name", device_map="
    auto")

image = Image.open("photo.jpg")
messages = [{"role": "user", "content": [
    {"type": "image"}, {"type": "text", "text": "Describe this image."}]}]

prompt = processor.apply_chat_template(messages, add_generation_prompt=
    True)
inputs = processor(text=prompt, images=image, return_tensors="pt").to(
    model.device)
```

```
# inputs now contains BOTH input_ids (text) and pixel_values (the image
    tensor)

out = model.generate(**inputs, max_new_tokens=200)
print(processor.decode(out[0], skip_special_tokens=True))
```

Shapes: the image arrives as `pixel_values` of shape `(batch, channels, height, width)` — the `(N, C, H, W)` convention from the PyTorch document. The processor resizes/normalizes the image to whatever the encoder expects (e.g. 384×384).

7 The Common Tasks

- **VQA (Visual Question Answering)** — ask a question about an image.
- **Captioning / description** — generate text describing an image.
- **OCR & document understanding** — read text, tables, and layout from scanned pages.
- **UI understanding** — interpret screenshots (the basis of computer-use agents).
- **Grounding** — locate or point to objects within an image.

8 Multimodal Embeddings and Cross-Modal Retrieval

Because CLIP/SigLIP put images and text in one space, you can search *across* modalities: embed a text query, then find the nearest *images* (or vice versa).

```
# Conceptually:
image_vecs = clip.encode_image(all_images)      # store these
query_vec  = clip.encode_text("a red bicycle") # embed the text query
# nearest image_vecs to query_vec are the matching pictures
```

This is what powers "find products that look like this" and text-to-image search.

9 Multimodal RAG (The Big Application)

Standard text RAG throws away diagrams, charts, and layout when it converts PDFs to plain text. **Multimodal RAG** keeps the visual content — and embedding images alongside text chunks has been reported to improve retrieval accuracy substantially (on the order of 25–40%) on visually rich corpora.

The image-indexing question (the one real decision): how do you make an image searchable?

1. Embed the **raw image** with CLIP — great for visual similarity, but can miss meaning obvious to a human.
2. Generate a **text description** with a VLM and embed that — enables text-to-image retrieval, but loses visual detail the caption omits.
3. **Both (hybrid)** — index the CLIP embedding *and* a VLM-generated-description embedding, and combine the signals. This is the recommended default.

The pipeline mirrors text RAG with extra steps at each stage: at *indexing*, extract text and visual elements (charts, photos, tables) and embed each with the right model; at *retrieval*, embed the query and find relevant elements across all modalities; at *generation*, assemble the retrieved text and images into a multimodal prompt and send it to a VLM.

10 Audio (Briefly)

Audio is just another modality with its own encoders:

- **Speech-to-text** — Whisper-style models transcribe audio to text (then you can use any text tool).
- **Text-to-speech** — generate spoken audio from text.
- **Audio embeddings** — models like wav2vec embed audio for search/classification, the audio analog of CLIP.

A common, robust pattern: transcribe audio to text with Whisper, then treat it as a normal text problem.

11 Generation vs Understanding

Two different multimodal directions:

- **Understanding** (the focus above) — image/audio in, text out (VLMs).
- **Generation** — text in, image/audio/video out. Image generation uses **diffusion** models (start from noise, iteratively denoise toward an image matching the prompt). Different architecture, same "shared conditioning on text" idea.

12 Choosing a Model

- **Proprietary (API)** — strongest reasoning over complex scenes; fastest to start; no infra.
- **Open weights** — choose for control, privacy, cost, and the ability to fine-tune/self-host.
- **Practical path** — prototype with a well-documented open VLM, then move to a stronger open model (the Qwen-VL / InternVL families are common production picks) if you need fine-tuning or data privacy.
- **Edge** — small multimodal models (1–10B) run on-device with some accuracy trade-off, for robotics, retail, and mobile.

13 Fine-Tuning a VLM (Awareness)

You can fine-tune a VLM with the same LoRA/QLoRA approach from the fine-tuning document, but the dataset is **image–text pairs** (e.g. image + question + answer). Custom VLMs typically need thousands to tens of thousands of labeled pairs — data is the bottleneck, not code.

14 Frontier (Awareness)

- **Native multimodal tool use** — newer models accept screenshots/document pages *directly as tool inputs* and interpret visual tool outputs (charts, webpage snapshots) in their reasoning.
- **Vision-Language-Action (VLA)** — models that see a camera feed plus an instruction and output robot actions; promising but still limited reliability.

15 The Checklist for Any Multimodal Task

1. Identify the modalities in and out (e.g. image in, text out → a VLM).
2. Decide hosted API (fast) vs open VLM (control/privacy/fine-tuning).
3. Build the multimodal prompt as a *list* of typed parts (image + text).
4. For search/RAG, pick your image-indexing strategy (CLIP, VLM-description, or both).
5. Watch image preprocessing (size/normalization) and the `(N,C,H,W)` shape.
6. For audio, consider transcribe-then-treat-as-text.

16 Practice Task

1. Send an image plus a question to a hosted multimodal API; get a description back.
2. Load an open VLM via `AutoProcessor`; run the same image and inspect the input keys (`input_ids` and `pixel_values`) and the image shape.
3. Use CLIP/SigLIP to embed 10 images and one text query; retrieve the best-matching image (cross-modal search).
4. Build a tiny **multimodal RAG**: for a few PDF pages with charts, store both a CLIP image embedding and a VLM-generated caption embedding; query in text and send the retrieved page image to a VLM to answer.
5. Bonus: wrap it behind FastAPI, serve the VLM (e.g. with vLLM, which supports many VLMs), and orchestrate retrieval with LangGraph — the same stack, now multimodal.